

RankSafe Prefix Cache for DINO-WM CEM Planning

Method. RankSafe Exact-Prefix Cache (RankSafe-EPC)

Motivation

DINO-WM’s test-time CEM planner evaluates several alternative action sequences from the same observed history. The visual and history computation before the action-dependent dynamics part is shared by these alternatives, but a straightforward implementation recomputes it for every branch. The challenge is therefore not only inference speed. Any reuse rule must keep the candidate-cost ordering unchanged, because that ordering determines the first action selected by the planner. A cache can save computation while silently changing that ordering and consequently changing closed-loop behavior, even when individual predictions appear close.

C^3 ache and X-Cache decide when to reuse computation using residual similarity or action-aware fingerprints, while WorldCache uses curvature and drift to trade rollout fidelity for speed. These methods show that computation reuse can be useful, but their validity signals do not state exactly which graph computation is independent of alternative actions. RankSafe-EPC addresses this gap in the DINO-WM Wall planner by isolating a prefix that is independent of the action, leaving the suffix unchanged, and directly testing whether candidate ordering and the first selected action stay the same.

Recent cache systems such as C^3 ache, X-Cache, and WorldCache make reuse practical but do not establish invariance of planner selection. The existing DINO-WM Wall checkpoint and local trajectories provide a fixed, reproducible setting for measuring graph instrumentation, exact-prefix reuse, native latency, and first-action agreement without retraining. A bounded implementation and matched evaluation fit within 2–4 A100s. The missing contract can therefore be tested now, while the claim remains provisional until native end-to-end and closed-loop evidence is obtained.

Earlier work stopped short for structural reasons. C^3 ache reuses residuals across smooth consecutive WAM chunks, rather than analyzing action independence in a planner graph. X-Cache uses approximate structure- and action-aware fingerprint gating, rather than certifying equality of a planner-relevant prefix across CEM candidates and preservation of the selected action. WorldCache targets rollout content fidelity, rather than the candidate ordering used by a sampling-based planner. EfficientVLA changes the represented computation through pruning, token selection, and feature caching, rather than isolating planner-preserving reuse at an action-dependency boundary. DINO-WM itself performs each counterfactual branch directly and does not expose a reusable prefix or a cache-validity test tied to candidate-cost and selected-action invariance.

If the gap is closed, shared-prefix reuse becomes a planner-level operation with an explicit condition for preserving CEM candidate ordering and the selected first action, instead of a similarity heuristic with an unknown downstream effect. It also gives systems such as C^3 ache and X-Cache a concrete boundary for distinguishing safe exact reuse from approximate cross-chunk reuse, while allowing native latency and closed-loop agreement to be measured together.

Method

M1_mechanism

Identify the exact action-independent boundary and reuse the shared prefix while preserving each action-conditioned suffix.

1. Use the DINO-WM Wall checkpoint in evaluation mode. Treat each action vector as an explicit input and list tensor boundaries in topological order from the observation/history encoder to the rollout head. For every boundary, compute automatic-differentiation Jacobians for every action vector in the current Cross-Entropy Method (CEM) population; mark a boundary element as dependent when any action-component derivative is nonzero, then combine the candidate masks elementwise. Choose the latest boundary whose combined mask is all zero, and store its module name, tensor shape, data type, device, and binary mask. If none is all zero, store ‘no_certified_boundary’ and evaluate this population through the full graph. Rebuild the mask whenever the observation/history or the CEM population changes. *The binary mask marks graph nodes at boundary b whose representation depends on the candidate action.*

$$M_b = \mathbf{1}\{\partial h_b / \partial a \neq 0\} \quad (1)$$

Why: The repeated work is the shared prefix, so an explicit dependency mask provides the exact condition for deciding whether that prefix can be reused.

2. For each observation/history, run the shared prefix once at the certified boundary with the same evaluation mode, data type, device, model weights, and preprocessing used by full recomputation. Store the boundary tensor with a cache key containing the observation/history identifier, checkpoint revision, boundary identifier, shape, data type,

and device. On a cache hit, present that tensor as read-only to each candidate-specific suffix, pairing each suffix call with its own candidate action while keeping every suffix parameter unchanged. Invalidate the entry when any key field changes, and reject in-place writes so one suffix cannot alter the value seen by another candidate. *When the boundary is action-independent, one cached prefix representation is valid for every candidate j .*

$$M_b = 0 \implies h_b^{(j)} = h_b^{\text{cache}} \quad \forall j \in \{1, \dots, K\} \quad (2)$$

Why: Exact reuse removes duplicated computation while leaving the action-conditioned part that determines counterfactual outcomes unchanged.

M2_validation

Guard candidate evaluation and test planner invariance, downstream decisions, and native latency.

- Run every action-conditioned suffix and retain its predicted observation sequence and per-horizon loss values. Use the planner-supplied rollout loss ‘ell’ against the matching reference observation at each horizon step, sum those losses into one scalar objective per candidate, and retain the candidate order. **【author decision: specify ‘ell’, the reference-observation source, normalization, and the horizon convention; these choices determine candidate ordering.】** Use the first candidate in the fixed population order as a deterministic canary: run it through the uncached full graph, compare its boundary tensor and final objective with the cached result using bitwise tensor equality (‘torch.equal’), and on any mismatch invalidate the current cache entry and recompute every candidate through the full graph. For the separate negative control, flatten ‘ M_b ’, apply a seeded uniform permutation that preserves the number of one entries, redraw the seed if the permutation is unchanged, disable fallback, and run the same suffix and objective pipeline; record the seed, corrupted mask, per-candidate objectives, and mismatch flag. *CEM ranks candidates by their rollout cost after the shared prefix and candidate-specific suffix are evaluated.*

$$J_j = \sum_{\tau=1}^H \ell(\hat{o}_{j,\tau}, o_{t+\tau}), \quad \pi = \text{argsort}(J_{1:K}) \quad (3)$$

Why: A gate could hide an error if it is contaminated by the result it is meant to protect. The canary and the matched corrupted-mask control expose that failure mode.

- Under identical CEM settings, use the same candidate population, random seed, initial history, checkpoint, preprocessing, rollout horizon, and action bounds for full recomputation, the valid-mask cached path (RankSafe), and the randomized-mask control. For each planner population, stable-sort candidates by scalar objective, breaking ties by candidate index; record rank disagreement as 1 only when the complete orderings differ, and record first-action agreement as 1 only when the first action vectors selected by the lowest-objective candidates are elementwise equal. Aggregate each binary metric over all evaluated populations by arithmetic mean and retain the per-population records. Run closed-loop episodes from identical initial states and seeds, then use the existing task evaluator to produce success outcomes. **【author decision: define the task success predicate, episode termination conditions, and outcome aggregation rule; without them closed-loop success is not reproducible.】** Measure native end-to-end latency from planner-call entry to the returned first action with the same warm-up and device-synchronization policy for all paths; retain every call duration and report the arithmetic mean and median. **【author decision: fix whether input preprocessing, synchronization, and environment interaction are included in the timing boundary; this choice defines native latency.】** *The mechanism check records rank disagreement and requires the selected first action to match the full recomputation.*

$$\Delta_{\text{rank}} = \mathbf{1}[\pi_{\text{cache}} \neq \pi_{\text{full}}], \quad a_{\text{cache}}^* = a_{\text{full}}^* \quad (4)$$

Why: The proposal concerns the planner’s selection quantity, so speed alone is not enough; ordering and action agreement must also be preserved.